

Ligero 論文詳解

Lightweight Sublinear Arguments Without a Trusted Setup

包括的な日本語訳・背景解説・図解

Ligero の構成スタック

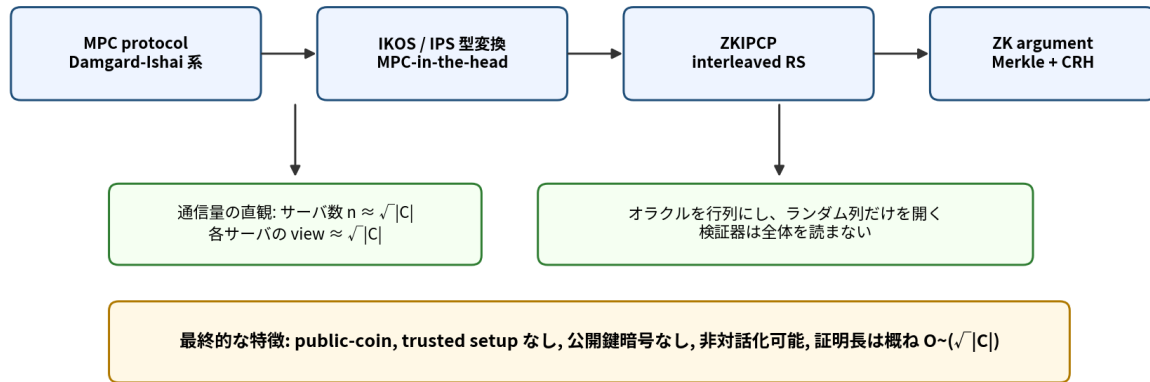


図0. 本書で解説する Ligero の全体像。

対象論文: Scott Ames, Carmit Hazay, Yuval Ishai, Muthuramakrishnan Venkitasubramaniam, “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup”. CCS 2017 版の拡張版。

本書は、原論文の全体構造、主要定義、プロトコル、定理、実装結果、付録の解析改善を日本語で再構成し、ZKP・PCP/IOP・MPC・Reed-Solomon 符号の背景を補った解説書である。数式と記号は原論文との照合を優先して一部英字のまま残している。

目次と読み方

Ligero は、公開鍵暗号や trusted setup を使わず、衝突困難ハッシュ関数と Reed-Solomon 符号を主材料にして、回路サイズ $|C|$ の平方根にほぼ比例する通信量を目指すゼロ知識 argument である。本書は初学者向けの背景から、プロトコルの実体と soundness 解析までを段階的に追う。

- 1. 第 1 章: Ligero の結論と設計思想。先に全体像を把握する。
- 2. 第 2 章: ZKP, SNARK, PCP/IPCP/IOP, MPC-in-the-head, Merkle tree, Reed-Solomon 符号の背景。
- 3. 第 3 章: 論文第 1-3 節の内容。MPC から ZKIPCP へ変換する考え方。
- 4. 第 4 章: 論文第 4 節の中核。interleaved RS code、線形・二次制約検査、算術/Boolean 回路、ゼロ知識化。
- 5. 第 5 章: 論文第 5 節。ZKIPCP をハッシュ/Merkle/Fiat-Shamir によって実際の ZK argument / zk-SNARK にする。
- 6. 第 6 章: 実装結果、証明長、計算時間、他方式との比較。
- 7. 第 7 章: 関連研究・未解決問題・付録の解析改善。

原論文の節	本書での対応	要点
1 Introduction / 1.1 Our Results	第 1 章, 第 3 章	目的、主結果、通信量 $O(\sqrt{ C })$ 、trusted setup 不要
2 Preliminaries	第 2 章	Merkle tree, ZK argument, IOP, IPCP
3 From MPC to ZKIPCP	第 3 章	MPC モデル、robustness/privacy、一般変換
4 Direct ZKIPCP Construction	第 4 章	interleaved RS, 線形/二次テスト、算術/Boolean 回路、ZK 化
5 From ZKIPCP to ZK	第 5 章	Merkle commitment, Fiat-Shamir、パラメータ最適化、多インスタンス
6 Implementation	第 6 章	NFLib, FFT, SHA256, 証明長・時間
7-8 and Appendices	第 7 章	後続研究、未解決問題、 $e < d/3$ / $e < d/2$ 解析、一般化テスト

表 0. 原論文と本書の対応。

第1章 エグゼクティブサマリー

Ligero の論文が示す中心的な成果は、「任意の NP 言明、より具体的には検証回路 C の充足性を、trusted setup なしで、通信量がおおむね $\sqrt{|C|}$ に比例するゼロ知識 argument として証明する」ことである。公開鍵暗号やペアリングではなく、衝突困難ハッシュ関数と Reed-Solomon 符号に基づくため、設計思想としては「軽量」「透明」「対称鍵プリミティブ中心」である。

原論文の informal theorem は、衝突困難ハッシュ関数が存在すれば、回路 C の充足性を証明する public-coin zero-knowledge argument が存在し、その通信量は $O(\sqrt{|C|})$ である、という形に要約できる。非対話化は Fiat-Shamir transform を random oracle model で用いることで実現される。

- 証明者は、回路評価の全ワイヤ値を少し冗長にした extended witness として並べ、それを $m \times \ell$ の行列に詰める。最適化では m と ℓ がどちらも roughly $\sqrt{|C|}$ になる。
- 各行を Reed-Solomon 符号化し、 m 個の codeword を列方向に束ねた interleaved RS codeword として扱う。
- 検証器は全体を読まず、ランダム線形結合と t 個のランダム列の開示だけで「符号に近いか」「線形制約を満たすか」「二次制約を満たすか」を検査する。
- ゼロ知識性は、ランダム codeword とランダム多項式によるブラインディングで確保される。
- 最終的な ZK argument では、巨大な oracle を直接送らず Merkle tree でコミットし、必要な列だけを authentication path とともに開く。

ZKP / SNARK の基本パイプライン



図1. ZKP / SNARK の一般的な処理パイプライン。Ligero は、IOP/IPCP 的な情報理論的検査を Merkle と Fiat-Shamir でコンパイルする系統に属する。

Ligero の性能上の見どころは、証明長が回路構造ではなく主にゲート数に依存する点である。原論文では、SHA-256 preimage の zero-knowledge 証明について、 2^{-40} soundness error で約 35KB 程度と述べている。また、30-bit prime field の回路では、40-bit security で約 260 万ゲート、128-bit security で約 2000 万ゲートを超えると、証明長が回路サイズそのものより小さくなる。

観点	Ligero の立ち位置
Trusted setup	不要。構造化 CRS や toxic waste を持たない。
暗号プリミティブ	衝突困難ハッシュ関数をブラックボックス的に使用。公開鍵暗号は不要。
対話性	public-coin。Fiat-Shamir により random oracle model で非対話化可能。
証明長	漸近的に $O(\sqrt{ C })$ 。実装では小-中規模回路で競争力がある。
証明者計算	主に RS encoding / evaluation / interpolation。FFT で高速化可能。
検証者計算	単一インスタンスでは回路読み込みや評価点計算の影響があるが、多インスタンス設定で amortized に改善。
安全性の型	ZKIPCP では統計的/完全ゼロ知識的な構成を行い、最終 argument はハッシュ仮定または random oracle model に基づく。

表 1. Ligero の要点。

第2章 背景知識

2.1 ゼロ知識証明・argument・SNARK

ゼロ知識証明は、証明者 P が「ある文 x が真である」ことを検証者 V に納得させる一方で、秘密 witness w については文の真偽以外を漏らさないためのプロトコルである。通常、完全性・健全性・ゼロ知識性の3性質で整理される。

性質	意味	Ligero での見方
Completeness	真の statement と正しい witness があれば、正直な P は V を受理させる。	正しい codeword と正しい制約多項式なら各テストが必ず通る。
Soundness	statement が偽なら、不正な P は高確率で V を騙せない。	interleaved RS 検査、線形制約検査、二次制約検査の失敗確率を合算して上界を出す。
Zero knowledge	V の view が simulator によって witness なしで再現できる。	ランダム codeword と多項式で、開示される列と線形結合を一様分布に近づける。
Argument	健全性が計算量制限付きの証明者に対する性質。	Merkle binding は衝突困難ハッシュ関数に依存する。
SNARK / zk-SNARK	Succinct non-interactive argument of knowledge. zk 付きなら zk-SNARK。	Fiat-Shamir で非対話化した Ligero は transparent な zk-SNARK と見なせる。

表2. ZKP 系の基本用語。

「proof」と「argument」の差は重要である。情報理論的な proof では、計算能力が無限の偽証明者でも通せない。一方、argument では偽証明者が多項式時間に制限され、暗号仮定の下で通せない。Ligero の中核 ZKIPCP は情報理論的な検査に近いが、実際に巨大 oracle を Merkle tree でコミットする段階で衝突困難ハッシュ関数に依存するため、最終的には argument となる。

2.2 trusted setup と透明性

多くの初期 zk-SNARK は、構造化された common reference string (CRS) を必要とした。CRS 生成時の秘密乱数、いわゆる toxic waste が漏れると、方式によっては偽証明の生成などに繋がる。このため、trusted setup ceremony が必要になる。Ligero の狙いは、このような secret trapdoor を伴う setup を避けることである。

Ligero では、公開パラメータとして有限体、評価点、ハッシュ関数などを合意すればよく、秘密 trapdoor はない。非対話版では random oracle model を用いるため、実装上は SHA256 などのハッシュ関数で challenge を導出する。

2.3 PCP, IPCP, IOP

PCP は、証明全体を読まずにランダムな少数箇所だけを読む検証モデルである。IPCP は、最初に PCP oracle を置き、その後に検証者と証明者が対話する拡張である。IOP はさらに一般的で、複数ラウンドで prover が oracle message を送る。Ligero の中核は ZKIPCP として記述され、最終的に Merkle tree と Fiat-Shamir によって普通の ZK argument へ変換される。

モデル	検証者の読み方	Ligero との関係
PCP	長い証明の一部だけを読む。	Kilian/Micali 型の SNARK の源流。
IPCP	初回に proof oracle、後続に対話メッセージ。	原論文の中核プロトコルは ZKIPCP。
IOP	複数ラウンドで oracle message をやり取	BCS/Fiat-Shamir 変換で非対話 argument

	り。	へ。
Linear PCP	proof vector の線形結合を少数取得。	多くの pairing-based SNARK の見方。 Ligero は公開鍵暗号を使わない。

表 3. PCP 系モデルの比較。

MPC-in-the-head から ZKIPCP への変換の直観

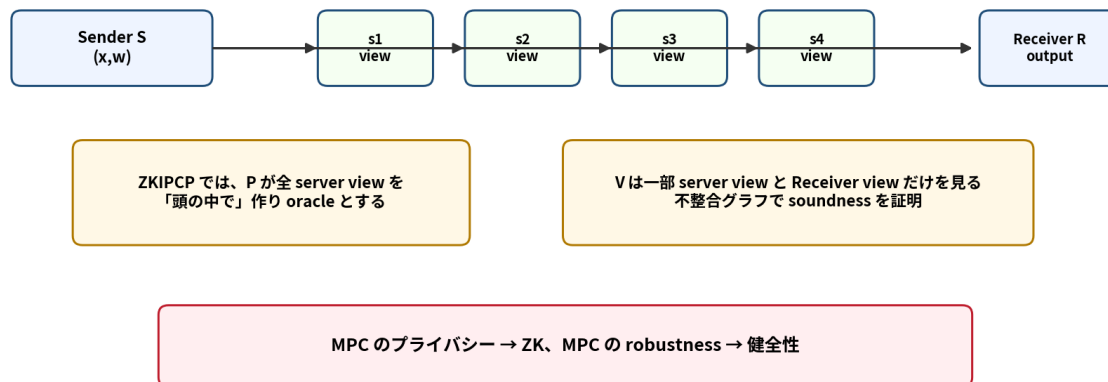


図 2. MPC-in-the-head の直観。MPC の仮想実行 transcript を証明 oracle とし、一部 view の開示で整合性を検査する。

2.4 Merkle tree と衝突困難ハッシュ

Ligero の最終 ZK argument は、ZKIPCP の oracle をそのまま送らない。証明者は oracle の各列や各エントリにコミットし、Merkle tree の root だけを先に送る。検証者が query した箇所について、証明者は値と authentication path を開示する。

Merkle tree の binding は、ハッシュ関数の collision resistance による。検証者は開示値から root までの path を再計算し、最初の root と一致するかを確認する。もし証明者が同じ root から異なる値を開けるなら、どこかで hash collision が見つかるはず、という論理である。

ZKIPCP から実際の ZK argument へ: Merkle commitment と Fiat-Shamir

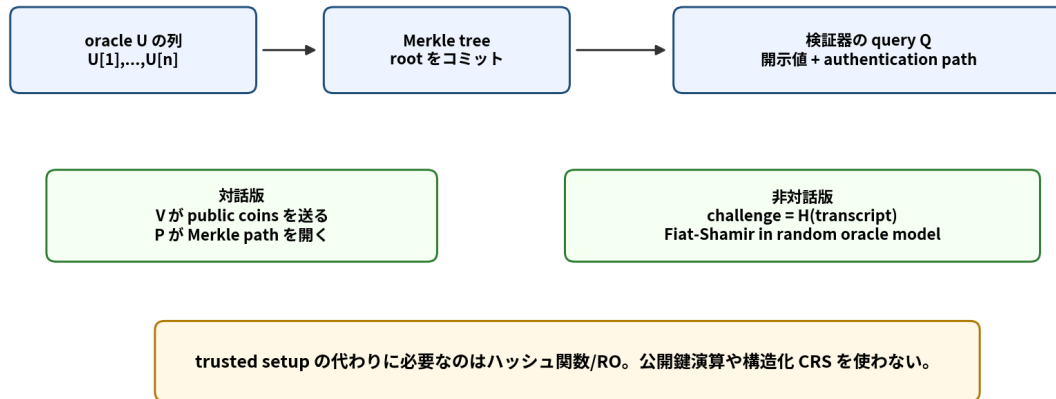


図3. ZKIPCP を通常の argument にするコンパイル。対話版は Merkle commitment、非対話版は Fiat-Shamir / random oracle を用いる。

2.5 Reed-Solomon 符号と interleaving

Reed-Solomon (RS) 符号は、低次数多項式 p を複数の評価点 $\eta_1, \dots, \eta_n$ で評価したベクトルからなる誤り訂正符号である。パラメータ $[n, k, d]$ は、codeword 長 n 、message 次元 k 、最小距離 $d=n-k+1$ を表す。Ligero では、witness を多項式の評価値として埋め込み、行ごとに RS 符号化する。

$$RS_{\{F, n, k, \eta\}} = \{ (p(\eta_1), \dots, p(\eta_n)) : \deg(p) < k \}$$

interleaved code L^m とは、 m 本の L -codeword を行に持つ $m \times n$ 行列である。ただし距離や query は列単位で見る。つまり $U[j]$ は j 番目の列、すなわち m 個の field elements を束ねた symbol である。この「列単位で開く」設計により、検証者は同じランダム列を使って複数の検査を同時に行える。

witness の $\sqrt{|C|} \times \sqrt{|C|}$ 行列化と interleaved Reed-Solomon 符号

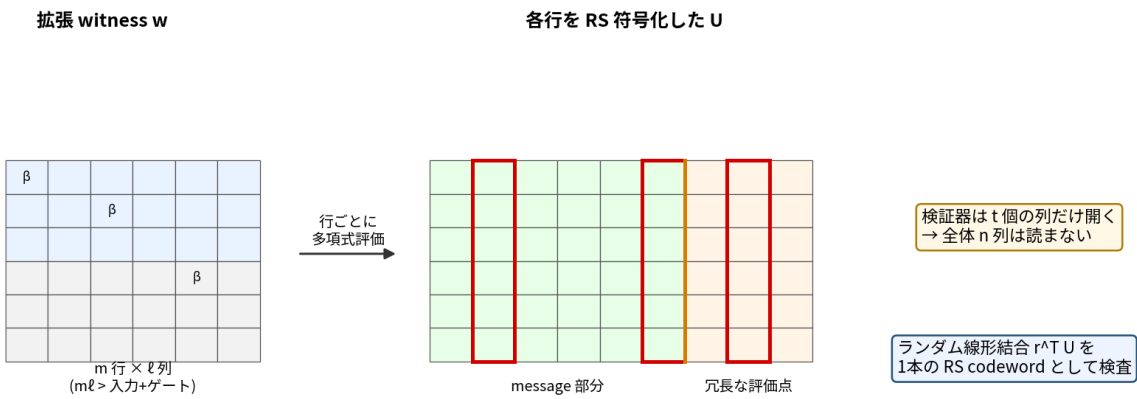


図 4. 拡張 witness の行列化と interleaved RS 符号。 $\sqrt{|C|}$ の行数と列数に近づけることが、通信量平方根化の直観。

第3章 論文の主結果と MPC→ZKIPCP 変換

3.1 論文の目的

論文の導入部は、アウトソース計算の検証という問題意識から始まる。計算を行う側が誤答を返す誘因を持つ場合、検証者が計算を再実行せずに正しさを確認できる argument が必要になる。Ligero は、この検証をゼロ知識性付きで、かつ通信量を回路サイズの平方根程度に抑えることを目指す。

既存の方向性として、GKR 型の doubly efficient interactive proofs、PCP/IOP 型、linear PCP + 公開鍵暗号型が紹介される。Ligero は、PCP/IOP 型の利点である trusted setup 不要・公開鍵暗号不要・ポスト量子安全性の期待を保ちつつ、従来より具体効率を改善することを狙う。

Ligero の構成スタック

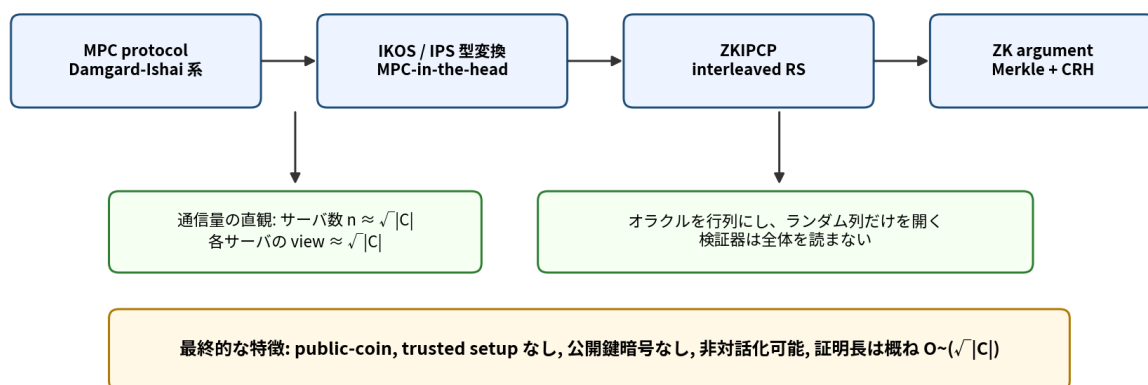


図5. Ligero は、MPC-in-the-head と interleaved RS code を使って軽量な ZKIPCP を作り、Merkle/Fiat-Shamir で ZK argument へ変換する。

3.2 Our Results の翻訳的要約

主結果は次のように言い換えられる。衝突困難ハッシュ関数が存在すると仮定すれば、任意の回路 C の充足性を証明する public-coin zero-knowledge argument が存在し、通信量は $O(\sqrt{|C|})$ である。public-coin であるため、Fiat-Shamir transform により random oracle model で非対話化できる。

原論文の主張	解説
Sublinear	通信量が検証回路サイズに対して線形未満。具体的には概ね平方根。
Simple/self-contained	重い PCP machinery ではなく、RS 符号と単純な線形/二次検査で構成。
Symmetric-key only	CRH/Merkle/RO のみ。ペアリングや群演算なし。
Public-coin	検証者メッセージが乱数だけなので Fiat-Shamir と相性が良い。
No trusted setup	非対話版でも構造化 CRS を必要としない。
Concrete efficiency	NFLlib/FFT/SHA256 実装で評価。
Multi-instance amortization	同じ回路の多数インスタンスで per-instance 通信と検証時間を改善。

表4. Our Results の日本語訳的要約。

なぜ平方根になるのか。MPC から見ると、 n 人のサーバの view を持ち、検証者は一部の view だけを見る。全通信量が回路サイズ $|C|$ 程度なら、サーバ数を $n \approx \sqrt{|C|}$ にすることで、1サーバあたりの view も $\sqrt{|C|}$ 程度になる。このトレードオフが ZKIPCP の oracle 行列に反映される。RS 符号の行列化でも、 $m \times \ell$ の m と ℓ を $\sqrt{|C|}$ 程度に選ぶことで、送る多項式長と開示列数のバランスが取れる。

informal theorem: CRH $\Rightarrow$ public-coin ZK argument for Circuit-SAT with communication $O(\sqrt{|C|})$.

3.3 MPC モデル

原論文第3節では、IKOS 変換の基盤として特殊な MPC モデルを導入する。登場人物は sender S 、 n 個の server $s_1, \dots, s_n$ 、receiver R である。 S は statement x と witness w を持ち、各 server に share/view を与える。servers はローカル計算し、最後に receiver にメッセージを送る。

要素	説明
Sender S	x と witness w を持つ。MPC の初期 share を server に送る。
Servers s_i	share を受け取り、ローカル計算する。簡略化されたモデルでは server 間通信を使わない。
Receiver R	各 server から出力メッセージを受け取り、 $R(x, w)$ の値を得る。
tp-privacy	高々 tp 個の server view と receiver view を見ても witness が漏れない。ZK に対応。
tr-robustness	不正 sender と高々 tr 個の server がいても、偽 statement を受理させにくい。soundness に対応。
consistent views	2つの view の送受信メッセージが互いに矛盾しないこと。ZKIPCP の検査対象。

表 5. MPC モデルの構成要素。

ZKIPCP 変換では、証明者がこの MPC 実行を「頭の中で」シミュレートし、server view の列を PCP oracle とする。検証者は receiver view とランダムに選んだ server view を開かせ、整合性を確認する。もし多数の不整合があればランダムサンプリングで見つかる。少数の不整合しかなければ、MPC の robustness により偽 statement は通らない。

Theorem 3.5 soundness: $\epsilon \leq (1 - tr/n)^{tp} + \delta(\kappa)$

第4章 Ligerio の中核: Direct ZKIPCP Construction

4.1 全体の流れ

第4節は、MPC 変換に依存しない self-contained な ZKIPCP の直接構成を与える部分で、論文の中心である。高レベルには、証明者は回路評価で得た全ワイヤ値を $m \times \ell$ 行列に詰め、各行を RS 符号化して U を作る。検証者は、ランダム線形結合と t 個のランダム列だけを使って、 U が妥当な符号語であり、かつ回路制約を満たすことを検査する。

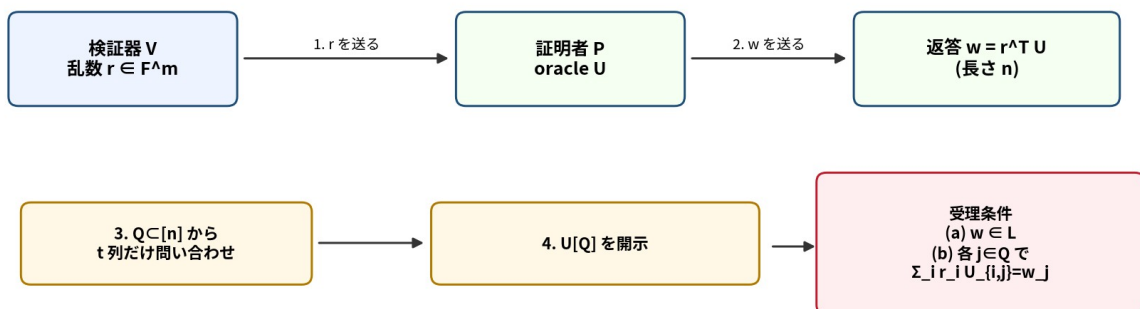
記号	日本語説明	役割
w	extended witness	入力値と各ゲート出力を並べたベクトル。
U	encoded extended witness	w を行列化し、各行を RS 符号化したもの。
m	witness 行列の行数	開示列 1 本あたり m 個の値を含む。
ℓ	witness 行列の列数	message 部分の長さ。 $m\ell$ が回路サイズを上回る。
s	circuit size	ゲート数。
k	message length	RS 多項式の次数上限に対応。
n	codeword length	RS 符号化後の列数。
d	codeword distance	最小距離。RS では $d=n-k+1$ 。
e	許容誤り列数	soundness 解析で「 e 列より遠い」を扱う。
t	query 数	検証器が開くランダム列数。
κ	security parameter	目標 soundness/security。
σ	repetition parameter	ランダム線形結合の反復数。
h	hash output length	Merkle/RO のダイジェスト長。

表 6. 原論文 Table 1 の日本語訳。

4.2 Test-Interleaved: interleaved linear code の検査

Test-Interleaved は、 m 本の行がすべて同じ線形符号 L に属するかを一度に検査する。検証者はランダムベクトル $r \in F^m$ を送り、証明者は $w = r^T U$ を返す。もし U の各行が L に属するなら、線形性により w も L に属する。さらに、検証者はランダムな t 列 Q を開かせ、返答 w と実際の列 $U[Q]$ の整合性を確認する。

Test-Interleaved の流れ



意味: 多数の行が正しい RS codeword かを、1本のランダム線形結合と少数列の整合性で同時に見る。

図6. Test-Interleaved の日本語図解。原論文 Figure 2 に対応。

soundness の核心は、 U が L^m から e より遠いなら、ランダム線形結合 $w=r^T U$ が L に近く見える確率は小さく、かつ w が不正ならランダム列サンプリングで高確率に検出できる、という点である。原論文の基本解析では $e < d/4$ で、拒否に失敗する確率はおおむね $(1-e/n)^t + (e+1)/|F|$ で上界される。

If $d(U^*, L^m) > e$ and $e < d/4$, then
 $\Pr[V \text{ accepts}] \leq (1 - e/n)^t + (e+1)/|F|$.

後の節では、RS 符号に対して $e < d/3$ まで解析を改善し、さらに後続研究の結果を取り込むことで $e < d/2$ に近いパラメータを使えることを議論する。これは concrete parameters を大きく改善する。

4.3 線形制約検査

Ligero の witness は、単に符号語であればよいわけではない。回路の配線と ADD gate の制約を満たす必要がある。これらは $Ax=b$ の形の線形制約として表現できる。単純にはランダム r を選んで $r^T A x = r^T b$ を確認すればよい。しかし x 全体を読めないため、証明者に対応する多項式 $q(\cdot)$ を送らせ、 q の係数と少数列の整合性で検査する。

Linear test idea:
 choose r ; check $(r^T A) x = r^T b$
 P sends $q(X) = \sum_i r_i(X) \cdot p_i(X)$
 V checks $\sum_c q(\zeta_c) = r^T b$ and $q(\eta_j)$ consistency for $j \in Q$.

この検査は「線形制約が1つでも破れていれば、ランダム線形結合で高確率に検出される」という Schwartz-Zippel 的な直観を持つ。さらに、証明者が偽の q を送れば、低次数多項式が一致できる評価点は限られるため、ランダム列で検出できる。

4.4 二次制約検査

MULT gate は $\beta_a \cdot \beta_b = \beta_c$ という二次制約である。Ligero では、 x, y, z という3つのベクトルを用意し、各 multiplication gate について x_j, y_j, z_j が入力2つと出力に対応するようにする。検査したい式は $x \odot y - z = 0$ である。より一般には $x \odot y + a \odot z = b$ の形で書かれる。

Quadratic test idea:
 $p_i(X) = p_i^x(X) \cdot p_i^y(X) + p_i^a(X) \cdot p_i^z(X) - p_i^b(X)$
 P sends $p\theta(X) = \sum_i r_i \cdot p_i(X)$
 V checks $p\theta(\zeta_c) = 0$ and consistency at sampled η_j .

回路の満足性を線形制約・二次制約に分けて検査する

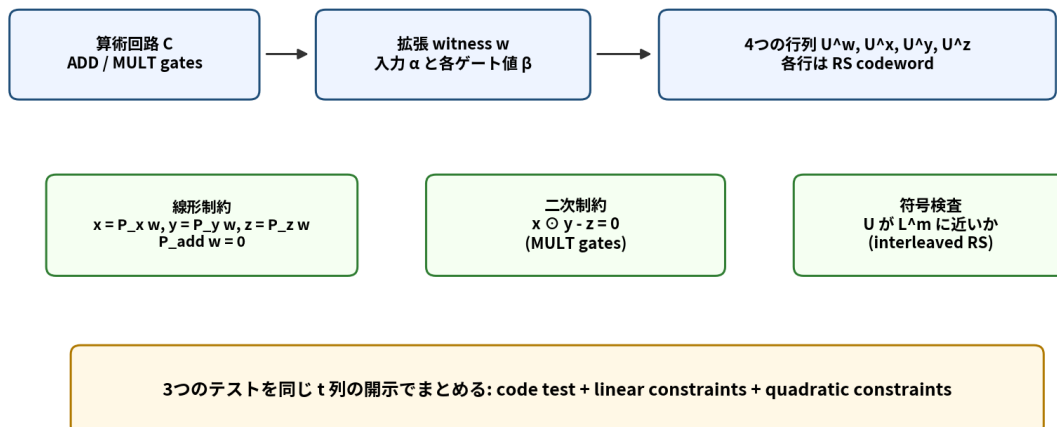


図7. 回路制約を線形・二次・符号検査に分解する。

線形制約検査と二次制約検査は、Test-Interleaved が「U は符号語に近い」と保証しているという条件付きで soundness を解析する。偽の多項式は低次数であるため一致点数が限られ、ランダム列を全部その一致点や誤り列に当てない限り検出される。

4.5 算術回路 IPCP

算術回路版では、有限体 F 上の ADD/MULTIPLY gate からなる回路 $C:F^{\wedge}\{n_i\} \rightarrow F$ を考える。証明者は witness α と各ゲートの値 β_i を並べて extended witness w を作る。各 multiplication gate について x,y,z に対応値を入れ、各 addition gate について $P_{\text{add}} w=0$ となるような線形制約を作る。

対象	制約化
入力値	w の先頭に $\alpha_1, \dots, \alpha_{\{n_i\}}$ を配置。
各ゲート出力	$\beta_1, \dots, \beta_s$ を w に追加。
MULT gate	$\beta_a \cdot \beta_b - \beta_c = 0$ 。 x,y,z に格納し、二次検査。
ADD gate	$\beta_a + \beta_b - \beta_c = 0$ 。 $P_{\text{add}} w=0$ として線形検査。
出力 gate	最終出力が 1 である制約を追加。

表 7. 算術回路充足性の制約化。

プロトコル $\text{IPCP}(C,F)$ は、 $U^{\wedge}w, U^{\wedge}x, U^{\wedge}y, U^{\wedge}z$ を縦に連結した $U \in L^{\wedge}\{4m\}$ を oracle とし、3つのテストを実行する。すなわち、(1) Test-Interleaved, (2) Test-Linear-Constraints-IRS, (3) Test-Quadratic-Constraints-IRS である。検証者は同じ t 列を開くため、列開示コストを共有できる。

IPCP soundness (representative):
 $\epsilon \leq (d+2)/|F| + (1-e/n)^t + 2((e+2k)/n)^t$

4.6 Boolean 回路への拡張

Boolean 回路を大きな有限体 F 上に埋め込むには、Boolean 0/1 を F の 0/1 に写す。各 witness 要素が bit であることは $\beta^2 - \beta = 0$ で強制できる。XOR と AND は補助 bit を用いて線形制約として扱える。

Boolean 制約	有限体上の表現
bit 性	$\beta^2 - \beta = 0$ 。0 または 1 のみを許す。
XOR	$b_1 + b_2 = b_3 + 2d$ 。 d は carry/補助 bit。
AND	$b_1 + b_2 = d + 2b_3$ 。 d は XOR 側の補助 bit に相当。
32-bit 加算 mod 2^{32}	$\sum 2^i a_i + \sum 2^i b_i = 2^{32} d + \sum 2^i c_i$ 。十分大きな標数が必要。

表 8. Boolean 回路の有限体埋め込み。

prime field を使う場合、XOR と AND の両方が算術的な制約コストを持つ。一方、 $\text{GF}(2^n)$ のような標数 2 の体では、XOR は加算として自然に扱えるが、FFT 実装や witness サイズとのトレードオフがある。論文の結論部でも prime field と characteristic 2 field の選択が今後の最適化点として述べられている。

4.7 ゼロ知識化

これまでの IPCP は soundness には有効だが、検証者は $r^T U$ や $U[Q]$ を見るため witness の情報を得てしまう可能性がある。ゼロ知識化では、開示される線形結合や局所列が witness から独立に見えるよう、ランダムな codeword や多項式を追加する。

ゼロ知識化: ランダムな codeword で線形結合をマスクする

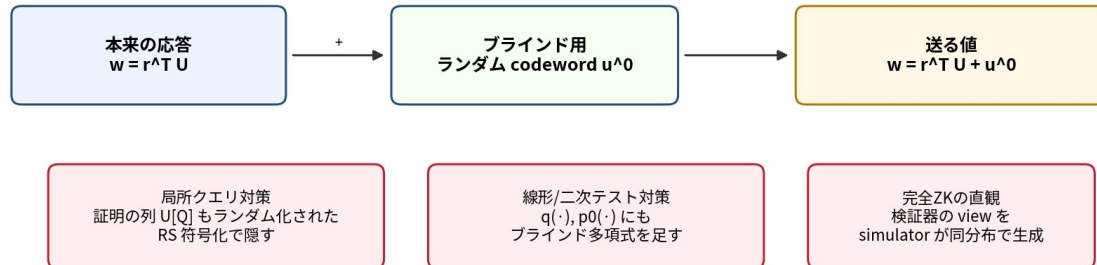


図 8. ゼロ知識化の中心アイデア。ランダム codeword による affine 版テスト。

interleaved code test では、 $w=r^T U$ の代わりに $w=r^T U+u^0$ を送る。 u^0 はランダムな L-codeword であり、正直な実行では $r^T U$ を完全に隠す。線形制約検査では、 $q(\cdot)$ に合計が 0 になるようなランダムメッセージを符号化したブラインド多項式を足す。二次制約検査でも、ゼロメッセージを符号化したブラインド多項式を足す。

最終 ZKIPCP の theorem では、 $k>\ell+t$ などの条件の下で、honest verifier perfect zero-knowledge が示される。証明は simulator を構成する形であり、検証者が見る多項式 $q_{\text{add}}, p0$, ベクトル v_h 、開示列を witness なしで同分布に生成できることを示す。

Ligero の soundness error は何を意味するか

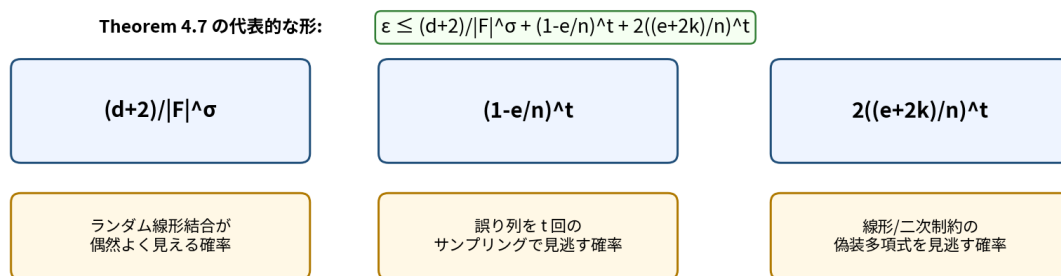


図 9. Final ZKIPCP の代表的な soundness error の構成要素。

Theorem 4.7 representative guarantee:
 Completeness = 1
 Soundness $\epsilon \leq (d+2)/|F|^\sigma + (1-e/n)^t + 2((e+2k)/n)^t$
 Perfect HVZK if $k > \ell+t$

第5章 ZKIPCP から ZK argument / zk-SNARK へ

5.1 対話版: CRH + Merkle tree

ZKIPCP では検証者が oracle にランダムアクセスできると仮定する。しかし現実のプロトコルでは、oracle を固定したことを証明者にコミットさせ、後で必要な箇所だけ開く必要がある。Ligero は、統計的 hiding commitment と Merkle hash tree を用いてこれを実現する。

証明者は oracle の各要素あるいは各列にコミットし、その全体を Merkle root に圧縮して検証者に送る。検証者が Q を選ぶと、証明者は $U[Q]$ と authentication paths を開示する。検証者は path を再計算し、root と整合するかを確認する。通信量には、ZKIPCP 自体のメッセージに加え、開示値と Merkle path の分が加わる。

5.2 非対話版: Fiat-Shamir in random oracle model

Ligero は public-coin である。つまり検証者のメッセージは「新鮮なランダム性」だけでよい。この性質により、Fiat-Shamir transform を適用できる。証明者は transcript の途中状態を random oracle に入力し、その出力を検証者 challenge として扱う。

非対話版では、証明者が各ラウンドの oracle/proof/message に Merkle commitment を置き、random oracle から導かれる challenge に対応して必要な path を含める。原論文は BCS16 の IOP から非対話 argument への変換を参照しつつ、Ligero の round-by-round soundness に対してよりタイトな解析を与える。

Non-interactive soundness sketch:
 $\epsilon_{NI} \leq T \cdot \epsilon_{IPCP} + O(T^2 \cdot 2^{-\kappa})$
where T = number of random-oracle queries by the cheating prover.

この式の意味は、Fiat-Shamir 後の敵対者が random oracle を何度も試せるため、元の対話プロトコルの soundness error が T 倍程度に増幅され、さらに hash collision / query collision に由来する $T^2 2^{-\kappa}$ 項が加わる、ということである。実装では random oracle output length を十分長く、たとえば 2κ bits に設定する。

5.3 通信量のパラメータ最適化

ZKIPCP をコンパイルした ZK argument の通信量は、主に3つのテストで送る多項式係数、開示される列、Merkle path からなる。原論文の式は次のように整理できる。

$$\text{communication} \approx [k\sigma + (k+\ell-1)\sigma + (2k-1)\sigma + t(4m+3\sigma)] \cdot \log|F| + t \cdot \log(n) \cdot h$$

ここで、 $k\sigma$ は code test、 $(k+\ell-1)\sigma$ は linear test、 $(2k-1)\sigma$ は quadratic test、 $t(4m+3\sigma)$ は開示列に含まれる view/field elements、 $t \log(n) h$ は Merkle decommitment の hash digest に対応する。

最適化の直観は、送る多項式側のコストと開示列側のコストを釣り合わせることである。典型的には $e=k$, $n=3k$ とし、 ℓ を $\sqrt{|C|}$ に比例する値に選ぶ。これにより、通信量は大きな field では $O(|F| \sqrt{|C|} \kappa)$ bits、小さな field では $O(\kappa \sqrt{|C|} |F|)$ bits と整理される。

$$\text{optimized } \ell \approx \sqrt{|C| \cdot \min(|F|, \kappa) / (5 \cdot \log_2(3/2))}$$

5.4 多インスタンス amortization

同じ回路 C に対して N 個の公開入力 x_i の充足性を同時に証明する場合、Ligero は witness を横方向に詰めることで amortized communication を改善できる。各 instance の witness w_i をまとめ、block j に全 witness の j 番目の値を置く。これにより、単一インスタンスで必要だった再配置のコストを削減できる。

複数インスタンス amortization の考え方

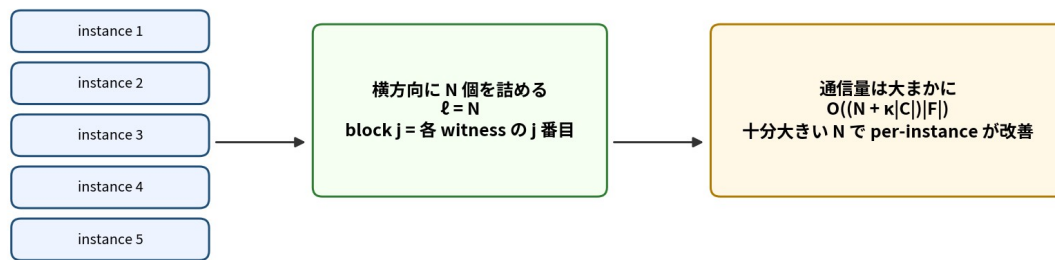


図10. 多インスタンス amortization。 N が十分大きいと per-instance 通信が大きく改善する。

原論文では、十分大きな field で総通信量が $O((N + \kappa|C|)|F|)$ になると述べている。 N が $\Omega(\kappa^2)$ 程度あれば、per-instance 通信量が回路サイズ s より小さくなるという主張も示される。検証時間についても、総 $O(s \log s + N \log N)$ field operations という形で amortized に sublinear な挙動が得られる。

第6章 実装と実験結果

論文の実装は、有限体演算に NFLlib を使い、評価点 η_i と ζ_j を roots of unity として選ぶことで interpolation と evaluation を FFT / inverse FFT で高速化している。実験環境は Amazon EC2 c6i.32xlarge、Intel Xeon 3.5GHz、128 cores、256GB RAM、衝突困難ハッシュ関数には SHA256 を使用した。

6 Implementation and Experimental Results

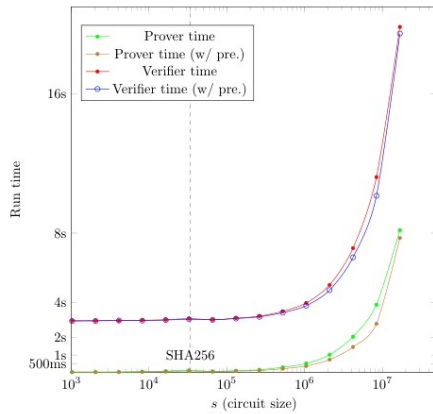


Figure 5: Prover and verifier running times for verifying a single instance of different circuit sizes.

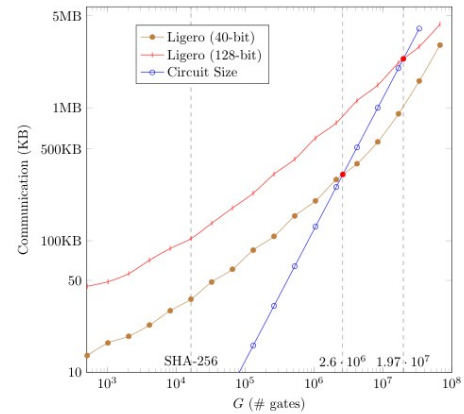


Figure 6: Proof lengths for proving a single instance of different circuit sizes.

図 11. 原論文 p.30 の Figure 5/6 再掲。左: prover/verifier running time、右: proof length。

単一インスタンス設定で packing factor を証明長最小に最適化すると、prover と verifier の計算量は $O(s \log s)$ field operations に比例する。field size の最適値は漸近的に $O(\log s)$ と見られるため、全体としては $O(s \log^2 s)$ と説明される。

小-中規模回路では、prover time の大きな部分が回路記述を読む preprocessing に費やされる。固定回路であればこの部分は事前計算できる。回路が大きくなるほど per-gate 効率は改善し、 10^7 gates では preprocessing を除いて約 500ns/gate、end-to-end で約 1.1 μ s/gate と報告されている。

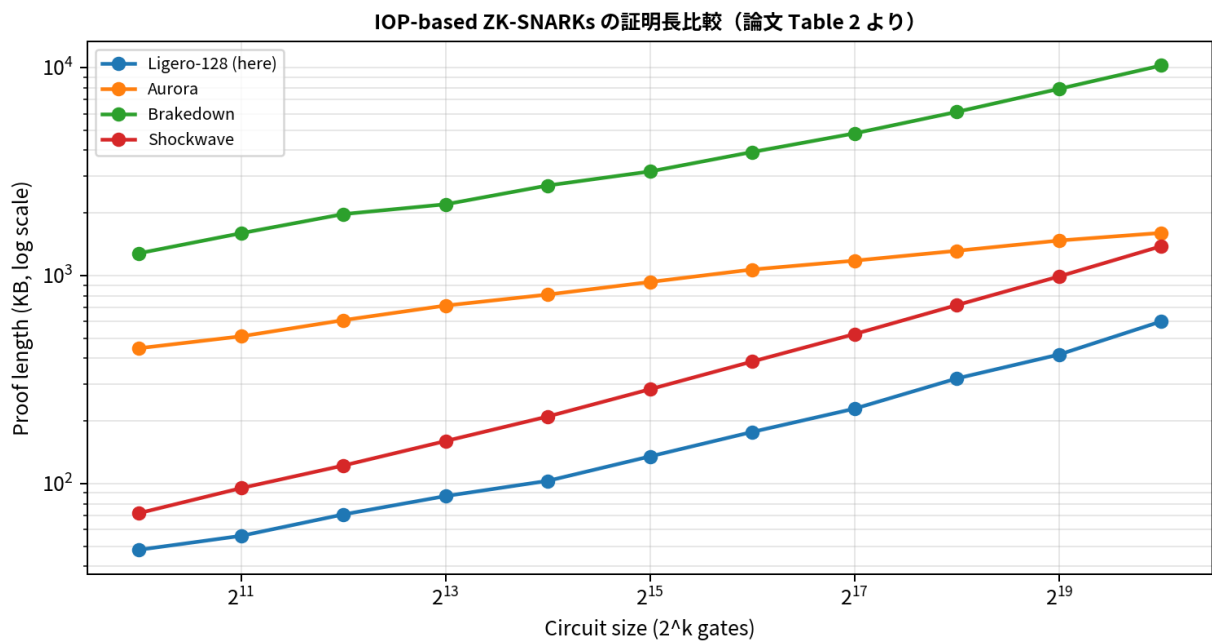


図 12. 原論文 Table 2 の証明長比較をグラフ化。Ligerio-128 (here) は小-中規模で短い証明長を示す。

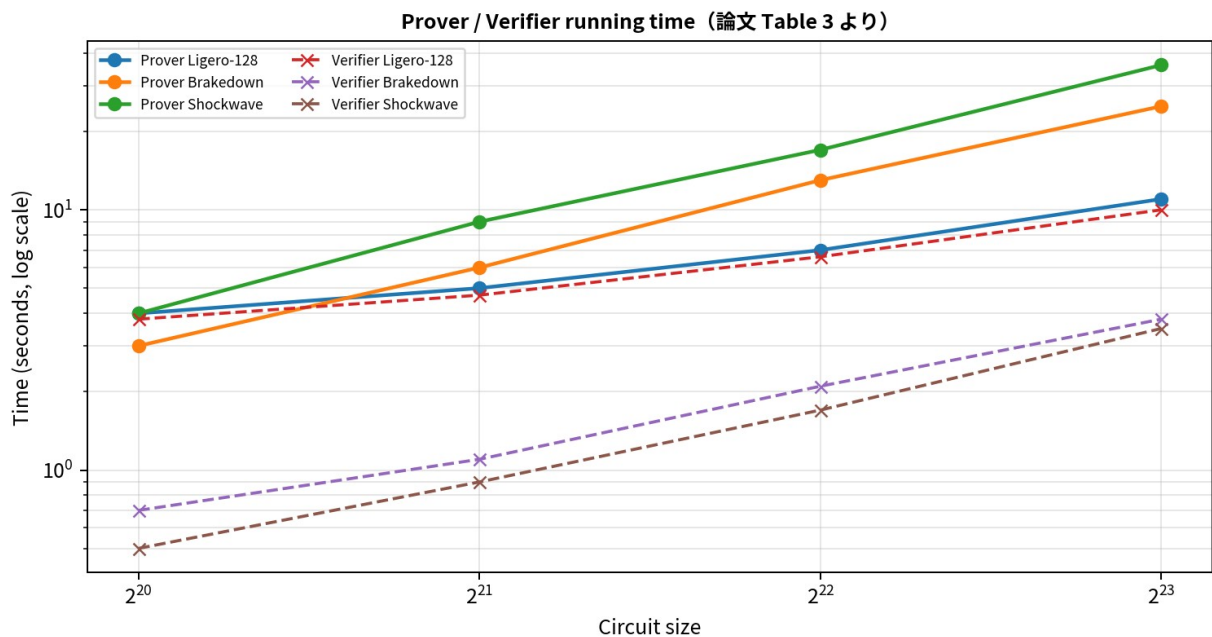


図 13. 原論文 Table 3 の prover/verifier time をグラフ化。機械環境差があるため論文自身も apples-to-apples でないと注記している。

Circuit size	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}
Ligerio-128 (here) KB	48	56	71	87	103	135
Aurora KB	447	510	610	717	810	931
Brakedown KB	1279	1597	1974	2200	2710	3165
Shockwave KB	72	95	122	160	210	284

表 9a. 証明長比較 (KB、原論文 Table 2 前半)。

Circuit size	2^{16}	2^{17}	2^{18}	2^{19}	2^{20}
Ligerio-128 (here) KB	177	229	320	417	602
Aurora KB	1069	1179	1315	1473	1603

Brakedown KB	3926	4824	6122	7899	10230
Shockwave KB	386	523	721	990	1384

表 9b. 証明長比較 (KB、原論文 Table 2 後半)。

Circuit size	2 ²⁰	2 ²¹	2 ²²	2 ²³
Brakedown prover	3	6	13	25
Shockwave prover	4	9	17	36
Ligero-128 prover	4	5	7	11
Brakedown verifier	0.7	1.1	2.1	3.8
Shockwave verifier	0.5	0.9	1.7	3.5
Ligero-128 verifier	3.8	4.7	6.6	10

表 10. 実行時間比較 (秒、原論文 Table 3)。

実験結果の読み方として重要なのは、Ligero が常に最短・最速という意味ではない点である。後続の STARK/Aurora/Brakedown/Shockwave などは、証明長・検証時間・証明者時間の異なるトレードオフを持つ。Ligero の価値は、非常に単純な interleaved RS ベースの構成で trusted setup なし・公開鍵暗号なしを実現し、特に小-中規模の回路や prover efficiency で強みを示した点にある。

第7章 関連研究・未解決問題・付録の役割

7.1 後続研究との関係

拡張版論文は、CCS 2017 版の後に現れた研究も概観している。zk-STARK、Aurora、Libra、Brakedown、Shockwave、Ligero++、Booligero など、透明・ポスト量子安全性が期待される proof system は急速に発展した。Ligero は後続方式に証明長の漸近では劣る場合がある一方、構成の単純性と prover efficiency、amortized setting の利点が強調される。

系統	特徴	Ligero との比較観点
STARK / FRI 系	低次数検査に FRI を使い polylog 証明長を狙う。	大規模では証明長・検証時間が有利になりやすい。
Aurora	R1CS 向け transparent zk-SNARK。	Ligero より証明長が小さい場合が多いが、Ligero は単純な平方根構成として位置づく。
Brakedown / Shockwave	線形時間 prover や異なる IOP 設計。	Ligero 拡張版は Table 2/3 で証明長・prover time を比較。
MPC-in-the-head signatures	署名などに応用される。	多くは回路サイズ線形にスケールし、Ligero の sublinear 性とは違う。
Booligero / Ligero++	Ligero の Boolean 特化・最適化。	Ligero の設計を土台に concrete proof size を改善。

表 11. 関連方式の位置づけ。

7.2 未解決問題

論文第7節は、IOP に関する未解決問題を挙げる。これらは Ligero だけでなく、透明・対称鍵ベースの succinct argument 全体に関わる。

- Boolean circuits に対して、negligible soundness error と constant computational overhead を両立する sublinear-query IOP を構成できるか。
- 時間・空間の両方を保つ complexity-preserving IOP を構成できるか。特に RAM 計算で T 時間 S 空間の witness 検証を、prover 側でほぼ同等の時間・空間で扱えるか。
- NP 全体の sublinear arguments を作る最小仮定は何か。collision-resistant hash functions より弱い one-way functions で足りるのか。

7.3 付録 A: $e < d/3$ の解析

付録 A は、interleaved code test の soundness 解析を $e < d/3$ に拡張するための補題を証明する。中心は、RS 符号からの距離について、affine line 上の点が「全て e -close」か「高々 d 個しか e -close でない」という二分性である。この補題により、ランダム線形結合が偶然 code に近く見える確率を $d/|F|$ で抑えられる。

Lemma 4.5 (RS, $e < d/3$):
 $\Pr[d(w^*, L) \leq e] \leq d/|F|$

7.4 付録 B: 一般化・affine 版テスト

付録 B は、 σ 個の線形結合による soundness amplification と、ゼロ知識化に必要な affine 版テストを形式化する。特に、random blinding vector $u \neq 0$ を oracle に追加し、 $r^T U + u \neq 0$ を検査しても soundness が保たれることを示す。線形制約・二次制約についても、extension field や repetition を使って $1/|F|^{\sigma}$ 型の項を得る。

付録Bの項目	役割
B.1 Generalized Interleaved Linear Code Testing	σ 個のランダム線形結合で soundness を増幅。
B.2 Affine Interleaved Linear Code Testing	ブラインド codeword u^0 を加えて ZK 化。
B.3 Generalized Affine Interleaved Linear Code Testing	affine 版と repetition を統合。
B.4 Affine Linear Constraint Testing	線形制約検査をゼロ知識化。
B.5 Generalized Quadratic Constraints	二次制約検査をゼロ知識化・増幅。

表 12. 付録 B の役割。

7.5 付録 C: soundness 解析の改善

付録 C は、IRS code test と線形/二次制約検査を独立に union bound するのではなく、連携して解析することで一部の項を改善する。直観的には、IRS test が通っているなら、誤り列を選んでいないという条件を活用できるため、線形/二次テストの $((e+k+l)/n)^t$ や $((e+2k)/n)^t$ の e を取り除ける。

Improved soundness form:

$$\varepsilon \leq (1-e/n)^t + ((k+l)/n)^t + (2k/n)^t + (n+3)/(2^{30})^\sigma$$

この改善は concrete parameters、つまり実際の proof length の削減に効く。Ligero の拡張版が旧 CCS 版より良い証明長を出している理由の一部は、実装の改善だけでなく、このような解析のタイト化にもある。

第8章 重要概念のミニ辞典

用語	説明
NP relation $R(x,w)$	statement x と witness w の関係。 w が存在すれば x は言語 L_R に属する。
Circuit satisfiability	回路 C に入力 α を入れて $C(\alpha)=1$ となる α が存在すること。
Arithmetic circuit	有限体上の ADD/MULTIPLY gate で表す回路。
Boolean circuit	AND/XOR/NOT などの bit 回路。有限体上に埋め込む必要がある。
Extended witness	入力 witness と各ゲート値を並べた冗長な witness。
Reed-Solomon code	低次数多項式の評価値列からなる符号。低次数性と誤り訂正距離を利用。
Interleaved code	複数 codeword を行列の行として並べ、列単位で見る符号。
Random linear combination	多数の行/制約を1本のランダム結合に圧縮して検査する技法。
Schwartz-Zippel intuition	非ゼロ多項式がランダム点でゼロになる確率は次数/体サイズ以下という直観。
Merkle tree	多数の値に1つの root でコミットし、log サイズの path で一部だけ開くデータ構造。
Fiat-Shamir transform	public-coin 対話プロトコルの challenge を $\text{hash}(\text{transcript})$ で置き換え非対話化する技法。
Random oracle model	ハッシュ関数を理想的なランダム関数として扱う安全性モデル。
Soundness error	偽 statement を受理してしまう確率の上界。
Honest-verifier ZK	正直に乱数を選ぶ検証者に対して witness を隠す ZK。
Perfect ZK	実行 view と simulator output が完全に同分布。統計距離すらない。

表 13. Ligerio を読むための用語集。

第9章 原論文の節別・翻訳的詳解

ここでは原論文の各節について、主張・定義・プロトコルの意味を日本語でまとめる。原文の数式番号や theorem/lemma 番号は、照合しやすいように概ねそのまま残す。

9.1 Abstract

要旨は、Ligero が「NP に対する単純なゼロ知識 argument」であり、通信量が検証回路サイズの平方根に比例することを宣言する。方式は任意の衝突困難ハッシュ関数に基づける。また random oracle model では Fiat-Shamir により非対話化でき、trusted setup や公開鍵暗号を必要としない concrete efficient zk-SNARK になる。技術的には、IKOS の MPC→ZK 変換を最適化し、Damgard-Ishai の MPC protocol の変種に適用したものと説明される。さらに、interleaved Reed-Solomon codes に基づく単純な zero-knowledge interactive PCP と見ることができる。

要旨の最後では、SHA-256 preimage の証明で 2^{-40} soundness error の通信量がおおよそ 35KB、 2^{-40} では約 300 万ゲート以上で回路サイズより通信量が小さくなること、改良解析により小-中規模回路で後続の post-quantum ZK-SNARKs より良い proof length/prover time を示すことが述べられる。

9.2 Section 1: Introduction

導入部の翻訳的要点は、検証可能計算とゼロ知識を組み合わせる動機である。証明者が大きな計算を実行し、検証者がその正しだけを安く確認したい場合、回路 C の充足性を sublinear communication で証明したい。Ligero は public-coin であるため、非対話化の道筋が自然にある。

関連する3つの流れとして、(1) GKR 型 doubly efficient interactive proofs、(2) PCP/IPCP/IOP、(3) linear PCP + homomorphic public-key cryptography が整理される。Ligero は (2) に近いが、重い PCP 機構を直接使うのではなく、MPC protocol と IKOS 変換を組み合わせる。

9.3 Section 2: Preliminaries

予備知識では、回路サイズ $|C|$ 、衝突困難ハッシュ関数、Merkle tree、zero-knowledge argument、IOP、IPCP、ZKIPCP を定義する。Ligero を実装するうえで特に重要なのは、Merkle tree が長い oracle を短い root に圧縮し、必要箇所だけ $O(\log n)$ 個の hash で開ける点である。

IPCP の定義では、prover は最初に PCP oracle π を計算し、その後 verifier と対話する。verifier は π 全体を読むのではなく q 個の bit/symbol だけを読む。Ligero の中核では、この oracle が interleaved RS codeword 行列 U である。

9.4 Section 3: From MPC to ZKIPCP

この節は、一般原理として「安全な MPC protocol から ZKIPCP を得る」ことを説明する。MPC の privacy は、ランダムに開かれる少数 view から witness が漏れないことを保証する。MPC の robustness は、偽 statement では receiver が 1 を出せないことを保証し、ZKIPCP の soundness に変換される。

不正証明者に対しては、server view と receiver view の不整合グラフを考える。不整合 edge が多ければ verifier がランダムに開いた view に当たりやすい。不整合 edge が少なければ、その少数 server を corrupt した MPC adversary を作ることで robustness に反する、という reduction が行われる。

Protocol Π_{ZKIPCP} simplified:

1. P simulates MPC and commits server views as oracle.
2. V sends public random string r .
3. P sends receiver view.
4. V samples Q of tp servers and opens their views.
5. V accepts iff opened views are consistent with receiver view.

9.5 Section 4: Direct ZKIPCP Construction

この節の翻訳的理解は「回路充足性を RS 符号の行列検査に落とす」ことである。まず符号距離や Decode_{ζ} を定義し、message 部分を ζ 点での評価値、codeword 部分を η 点での評価値として扱う。次に Test-Interleaved、線形制約検査、二次制約検査を順に作り、最後に算術回路と Boolean 回路の充足性に適用する。

線形制約検査と二次制約検査は、いずれも「巨大な制約集合をランダム線形結合で1つの多項式条件に圧縮する」形をしている。Verifier は P から多項式係数を受け取り、message 点 ζ での条件と、sampled codeword 点 η_j での整合性をチェックする。

ゼロ知識化は affine 版テストによって行われる。通常の線形結合にランダム codeword を足し、線形/二次テストの多項式にもブラインド多項式を足す。この処理により、検証者が受け取る view は simulator が witness なしに作れる同分布となる。

9.6 Section 5: From ZKIPCP to ZK

この節は実運用可能な argument へのコンパイルを扱う。対話版では、oracle へのランダムアクセスを Merkle commitment で実現する。非対話版では、public-coin challenge を random oracle に置換する Fiat-Shamir transform を使う。

また、パラメータ選択により通信量が sublinear になることを具体式で示す。重要なのは、 k, ℓ, m, n, t, σ のバランスである。 $n=3k, e=k$ のように設定し、 ℓ を $\sqrt{|C|}$ 程度に選ぶことで、多項式係数送信量と列開示量が釣り合う。

9.7 Section 6: Implementation

実装節は、FFT を使った RS encoding の高速化、SHA256 を使った random oracle/Merkle、30-bit prime field 上の算術回路評価、証明長と running time のグラフを報告する。Ligero は回路 topology に依存せず witness size、つまり主にゲート数に依存する。

原論文の比較では、拡張版 Ligero-128 は Table 2 の範囲で、Aurora, Brakedown, Shockwave と比べ短い proof length を示す。ただし比較は R1CS/算術回路の違いや実験機械の違いがあり、論文自体も apples-to-apples ではないと注意している。

9.8 Section 7-8 and Appendices

関連研究節は、後続の IOP-based transparent SNARKs、MPC-in-the-head signatures、Boolean 特化版 Ligero、Ligero++ などを整理し、未解決問題を提示する。結論では、prime field と characteristic 2 field の選択、部分評価による verifier 改善、GPU FFT、uniform circuits の利用などの実装最適化余地を挙げる。

付録は単なる補足ではなく、concrete efficiency に直結する。A は $e < d/3$ 解析、B は generalized/affine tests、C は IRS test と制約 test を一緒に見た soundness 改善を扱う。

第10章 Ligerio を実装・研究で読む際のチェックリスト

- 回路表現を確認する: arithmetic circuit なのか Boolean circuit なのか、R1CS に変換するのかで witness size と field choice が変わる。
- 有限体を選ぶ: FFT のために十分大きな 2-power roots of unity が必要になる場合がある。prime field と $GF(2^n)$ では XOR/AND のコストが異なる。
- m, ℓ, k, n を決める: $m\ell > \text{witness size}$ 、 $k > \ell + t$ 、 n と k の距離 $d = n - k + 1$ 、 e の制約を同時に満たす。
- soundness error を分解して見る: code test の項、query miss の項、linear/quadratic test の項、Fiat-Shamir 後の T 倍項を別々に見積もる。
- Merkle tree を列単位で設計する: 同じ query index で複数 oracle を開くため、column hashing/path pruning が証明長に効く。
- preprocessing を分離する: 固定回路では回路読み込みや配線行列の準備を前処理化できる。
- 多インスタンスを検討する: 同じ回路で多数 witness を証明する用途では Ligerio の amortization が特に有利になる。

Ligerio は、現代の proof system と比較すると「最短証明長のチャンピオン」というより、「非常に単純な代数的検査で trusted setup 不要・公開鍵暗号不要・平方根通信を達成する、透明 SNARK 系の重要な基礎構成」と捉えると理解しやすい。特に、interleaved RS code に対するランダム線形結合テストは、後続の IOP/encoded IOP の設計を理解するうえでも有用である。

参考文献・原論文との対応

主対象: Scott Ames, Carmit Hazay, Yuval Ishai, Muthuramakrishnan Venkitasubramaniam, “Ligerio: Lightweight Sublinear Arguments Without a Trusted Setup”. Extended version of CCS 2017 paper.

本書内では、原論文に現れる主要参考文献名を本文中の文脈で説明した。詳しい bibliographic data は原論文末尾の References を参照。

参照	本書での役割
IKOS07/09	MPC から ZK proof / ZKIPCP へ変換する基礎。
DI06	Ligerio が土台にする efficient MPC protocol。
BCS16	IOP を random oracle/Merkle で非対話 argument にする変換。
FS86	Fiat-Shamir transform。
Merkle89	Merkle hash tree。
BBHR19 / zk-STARKs	transparent proof system の並行・後続研究。
BCR+19 Aurora	R1CS 向け transparent zk-SNARK。Ligerio との比較対象。
BFH+20 Ligerio++ / GSV21 Booligerio	Ligerio 系の後続最適化。

表 14. 主要参考文献の役割。